

Clarifying the status of the “C headers”

Abstract

We propose to move the specification of “[depr.c.headers] C headers” from Annex D into the main document, and changing those headers’ status from “deprecated” to an explicitly discussed state “for foreign-language interoperability only”.

Contents

1. [Revision history](#)
2. [Motivation](#)
3. [Discussion and Feedback](#)
4. [Impact on the Standard](#)
5. [Proposed wording](#)

Revision history

- this version: initial version

Motivation

The C++ standard library includes large parts of the standard library of ISO C: where ISO C provides a header `<name.h>`, C++ provides a header `<cname>` with names declared in namespace `std`. For example, C++’s `<cstdint>` defines a type alias `std::size_t`. Additionally, C++ has always shipped a second set of headers, called “C headers”, which are also of the form `<name.h>`, just like in C — but note that these are simply part of the C++ standard library and thus of C++. These headers define the names in the global namespace. (Additionally, either class of headers is optionally allowed to also declare names in the

respective other namespace. This cannot be portably relied upon, however, it does have the effect of making the names in question reserved in the global namespace.)

Ever since C++ has been an ISO standard, the C headers have been “deprecated”, which in ISO parlance means “no longer in use”, “discouraged”, or “subject to future removal”. However, this is inappropriate: The C headers are a necessary part of the C++ ecosystem. What is missing from the status quo is a nuanced discussion of what the headers are and are not for. We will lay out these nuanced points now.

The principal purpose of the C headers is for *interoperability with ISO C*, and with any other systems that use C’s de-facto ABI and linkage conventions. For example, in C++ the header `<stddef.h>` does not just define the same names as the header of the same name in C, but the names actually mean the same, and this extends to the names defined by `<cstddef>`. For example, a C++ program may include that header, declare a function `extern "C" void f(std::size_t)`, and call that function. The function can be defined by a C translation unit, compiled with a C compiler, which defines `void f(size_t)` (after including `<stddef.h>`). This is not explicitly guaranteed to work by either language standard, but is universally relied upon by real ecosystems. The existence of “C language linkage” suggests that such interoperation is intended to work.

Finally, this allows us to articulate the role of the “C headers” in C++: These headers are necessary for polyglot interoperability code, i.e. code that is required to be at once valid C++ and also valid C. Typically, such code consists of header files which are included from both languages, and functions defined in those headers may be defined either in C++ or in C translation units. The headers are *not* useful in code that is *only* required to be valid C++. Therefore, the C headers should be provided by the C++ standard library as a fully-supported, not deprecated part, *but* they should also be discouraged for use in code that is not polyglot interoperability code.

Within pure C++ code, the use of the “proper” `<cname>` headers has several advantages:

1. Consistency and simplicity: Every C++ standard library name is in namespace `std`.
2. Some facilities are only available in a `<cname>` header: special maths functions, `std::byte`, `std::nullptr_t`. It would be burdensome to add an exception to explain how to get those facilities.

3. As a visual aid: It is useful to develop a habit that “things in a namespace require a header inclusion”. Typing “`std::something`” reminds the author (or reviewer) to check for the relevant header inclusion. By comparison, it is easier to miss an unqualified “`size_t`” and spot a missing header inclusion.

Note that the headers `<stdbool.h>` and `<stdalign.h>` are entirely trivial and only exist because their ISO C counterpart is necessary in C programs. Pure C++ code has no need for these headers, and so there is no corresponding `<cname>` header. Yet the existence of the trivial interop `<name.h>` header is critical.

Discussion and Feedback

The fate of the “C headers” has been brought up in [P2139](#). During a telecon discussion, LEWG and that paper’s author agreed that the topic should be handled by a separate paper, and there was general consensus that the current, deprecated status of the headers is wrong. Feedback to a draft of this proposal was generally positive, but included three noteworthy arguments that we would like to record here.

The first argument was that we should take a wider look at C interoperability, and perhaps even consider language support for addressing the way in which the “C names” are declared in two namespaces. The author of P2139 has also expressed interest in exploring farther-reaching language changes to support interoperability. However, this is out of scope for the current proposal, in which we only want to make one small change to elevate the headers from their current deprecated status to a place where they are a regular, normative feature, explicitly encouraged for polyglot code, and discouraged for pure C++ code that never needs to be valid C.

The second argument from a long-time committee member was that we should not overturn existing committee decisions without new information. In this author’s opinion, however, which seems to resonate with many other committee members, that original decision (which goes back to the first standard, C++98) was never quite perfect in the first place, and many have since come to regard the deprecated status of the headers as a mistake. From observing the situation for several years, we have reached the conclusion that the subject has simply been too marginal and irrelevant to motivate action, and we simply want to pick up that unfinished business with this proposal.

The third argument was that C and C++ should look as similar as possible (which would speak for using the unqualified names from the “C headers”). This author does not share that opinion, and instead believes that users should develop language fluency for each language *as appropriate for that language*. When writing C++, C++ sensibilities should come into play; it is not helpful to C++ code health if C++ code is made to look like a different language. The aforementioned connection between reading a namespace qualification and remembering to check for header inclusion is a valuable part of C++ fluency that simply does not apply to C, and it should not be abandoned because of C. When writing polyglot C and C++ code, one is addressing *two audiences* and needs fluency in *both* their languages, rather than hypothetical fluency in one hypothetical “common subset”.

Finally, we would like to repeat that this proposal is an overall improvement for everybody: Currently, the “C headers” are deprecated for everyone, meaning that they are notionally at the risk of future removal from the Standard (which we know will not happen). Under this proposal, the headers become normatively encouraged for polyglot code, and discouraged for pure C++ code. In light of other proposals in the area of C interoperability, it should be appreciated that the proposal makes the headers normatively encouraged for code that is intended to be both valid C++ and valid C. Discouragement is only given for code that is *only* C++.

Impact on the Standard

There is no change to the actual content of the Standard. This proposal makes the C headers no longer deprecated, so there is no formal threat of future removal. The effective discouragement to use the C headers in pure C++ code is now spelled out explicitly as normative discouragement.

Proposed wording

Move [depr.c.headers] to Clause [17, support]

~~D.11~~17.?? C Headers [~~depr~~support.c.headers]

~~D.11.1~~17.??.1 General [~~depr~~support.c.headers.general]

For compatibility with the C standard library, the C++ standard library provides the *C headers* shown in [Table 149, depr.c.headers]. The intended use of these headers is for interoperability only. It is possible that C++

source files need to include one of these headers so as to be valid ISO C. Source files that are not intended to also be valid ISO C should not use any of the C headers.

[Note: The C headers either have no effect, such as <stdbool.h> and <stdalign.h>, or otherwise the corresponding header of the form <cname> provides the same facilities and assuredly defines them in namespace std. — end note].[Example: The following source file is both valid C++ and valid ISO C. Viewed as C++, it declares a function with C language linkage; viewed as C it simply declares a function (and provides a prototype).

```
#include <stdbool.h>    // for "bool" in C, no effect in C++  
#include <stddef.h>    // for "size_t"  
  
#ifdef __cplusplus     // see [cpp.predefined]  
extern "C"             // see [decl.link]  
#endif  
void f(bool b[], size_t n);
```

— end example]

Move the remainder of the subclause as is.